

Serviços de Monitorização Urbana para One Health: Uma Infraestrutura Distribuída em C#

Bruno Sousa al81033, Lucas Cerqueira al81828, Hugo Costa al81748

Github: <https://github.com/RafaelSousa29/ProjetoSD>

Licenciatura em Engenharia Informática

Sistemas Distribuídos - pl4

Universidade de Trás-os-Montes e Alto Douro

Vila Real, Portugal

Abstract

O presente relatório descreve a arquitetura e implementação do Trabalho Prático nº 2 de Sistemas Distribuídos. Mantendo o enquadramento no paradigma One Health, o sistema integra comunicação Publish/Subscribe via RabbitMQ para ingestão assíncrona de dados ambientais, Remote Procedure Calls para pré-processamento e análise estatística, e sockets TCP para controlo e agregação. O conjunto resulta numa infraestrutura modular, com persistência em SQLite e dashboard web interativo.

1. Introdução

O objetivo central foi criar uma infraestrutura distribuída capaz de simular a recolha contínua de dados ambientais (temperatura, humidade, PM2.5, ruído, luminosidade e vídeo), o seu encaminhamento agregado via Gateway, e o armazenamento persistente no Servidor. Foram ainda implementadas funcionalidades extra, base de dados relacional SQLite, simulação de bateria do sensor e streaming de vídeo ponto-a-ponto, que são destacadas ao longo do documento. A Parte 2 do trabalho introduz três pilares arquiteturais: desacoplamento via Pub/Sub com RabbitMQ, delegação computacional via RPC e persistência estruturada com dashboard web, tornando o sistema escalável, resiliente e observável.

2. Arquitetura do Sistema

O sistema é composto por sete componentes distintos, cada um com responsabilidades bem delimitadas. A tabela seguinte resume os componentes e as suas funções principais.

Componente	Tecnologia	Função Principal
Sensor	C# / .NET	Gera medições, heartbeats, notificações e streams de vídeo
Gateway	C# / .NET	Valida sensores, agrega medições e encaminha lotes para o Servidor
PreProcessamentoRPC	Python	Valida e normaliza medições via RPC (porta 7000)
RabbitMQ	AMQP Broker	Distribui medições por tópicos (ex: TEMP, RUÍDO, S105:TEMP)
Servidor	C# / .NET	Recebe lotes, persiste dados em SQLite e aciona análise RPC
AnaliseRPC	Python	Calcula estatísticas e nível de risco via RPC (porta 7001)
Dashboard	HTTP / Web	Interface de visualização em localhost:8080

Tabela 1 — Componentes do sistema e respetivas funções.

O fluxo principal de informação segue a cadeia: Sensor → RabbitMQ → Gateway → Servidor → AnaliseRPC → Dashboard. Paralelamente, os sensores mantêm uma ligação de controlo TCP direta ao Gateway para envio de heartbeats, notificações de bateria e streams de vídeo.

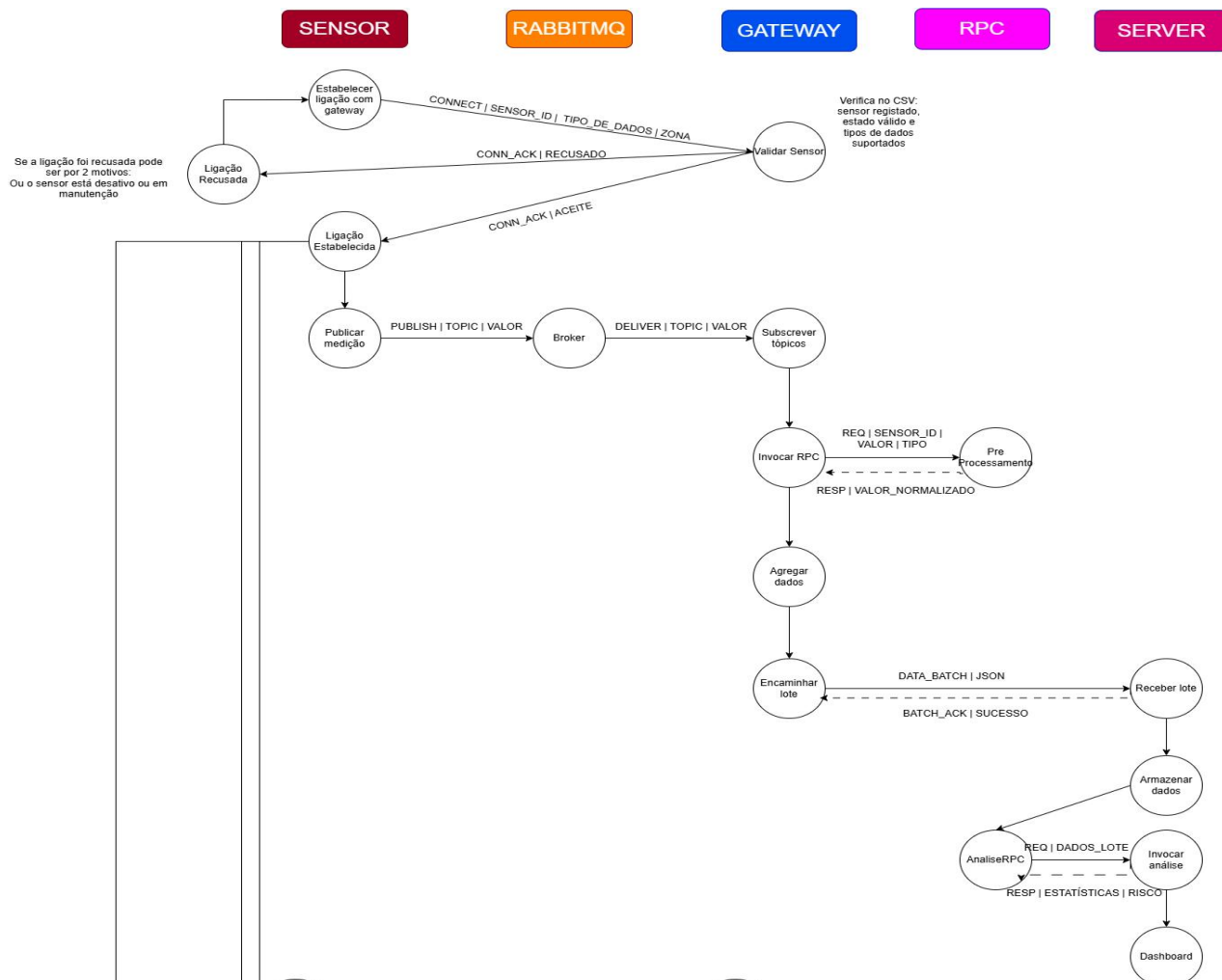


Figura 1 — Diagrama parcial do protocolo de comunicação (fluxo principal de dados). Os fluxos de controlo TCP — heartbeats, notificações de bateria, streaming de vídeo e desconexão — seguem o mesmo padrão de mensagens estruturadas entre Sensor e Gateway, e estão omitidos por razões de espaço.

3. Mecanismos de Comunicação Distribuída

A arquitetura combina três paradigmas de comunicação distintos, cada um selecionado de acordo com as necessidades de cada interação. A tabela seguinte apresenta o mapeamento completo.

Origem	Destino	Mecanismo	Porta / Canal
Sensor	Gateway	TCP (CONNECT, HEARTBEAT, NOTIFY, VIDEO)	5000,5001,5002,...
Sensor	RabbitMQ	Pub/Sub AMQP — publica medições por tópico	5672
RabbitMQ	Gateway	Pub/Sub AMQP — subscreve tópicos configurados	5672
Gateway	PreProcessamentoRPC	RPC via TCP	7000
Gateway	Servidor	TCP — envio de lotes JSON estruturados	6000
Servidor	AnaliseRPC	RPC via TCP	7001
Servidor	Browser	HTTP — dashboard web	8080

Tabela 2 — Mapeamento dos mecanismos de comunicação entre componentes.

3.1 Sockets TCP

Os sockets TCP são utilizados em dois contextos independentes. Na camada de controlo, cada Sensor estabelece uma ligação persistente ao Gateway para envio de mensagens estruturadas: **CONNECT** (registo inicial), **HEARTBEAT** (sinal de vida periódico), **NOTIFY** (alertas de bateria/carregamento) e **VIDEO** (stream de vídeo simulada). Na camada de agregação, o Gateway envia lotes JSON estruturados ao Servidor através de uma ligação TCP na porta 6000, sem qualquer intermediário.

3.2 Remote Procedure Call (RPC)

O RPC é utilizado para delegar processamento especializado a serviços externos, mantendo o código dos componentes principais focado na orquestração. O Gateway invoca o `PreProcessamentoRPC` (porta 7000) para cada medição recebida, obtendo o valor normalizado antes de incluir na agregação. O Servidor invoca o `AnaliseRPC` (porta 7001) automaticamente após receber cada lote, obtendo estatísticas e o nível de risco calculado. Em ambos os casos, a comunicação é síncrona e orientada a pedido-resposta, com tratamento de falhas por timeout.

3.3 Publish/Subscribe com RabbitMQ

O RabbitMQ atua como broker central para a ingestão de medições ambientais. Os Sensores publicam medições em tópicos hierárquicos (por exemplo, `TEMP`, `RUIDO`, `S105` ou `S105:TEMP`), permitindo que o Gateway subscreva seletivamente por tipo de dado ou por sensor. Este desacoplamento elimina dependências diretas entre Sensores e Gateway, tornando a adição de novos sensores ou tipos de dados transparente para o restante sistema.

4. Funcionamento Implementado

4.1 Ciclo de Vida dos Dados

Cada medição segue o seguinte percurso: o `DataGenerator` produz um valor simulado (temperatura, ruído, etc.) que o Sensor publica no RabbitMQ sob o tópico correspondente. O Gateway, que subscreve os tópicos relevantes, recebe a medição e invoca o `PreProcessamentoRPC` para a validar e normalizar. O valor tratado é então acumulado num lote. Quando o lote atinge o limiar configurado, é serializado em JSON e enviado ao Servidor via TCP. O Servidor persiste os dados em SQLite e em ficheiros organizados por tipo, invoca o `AnaliseRPC` e atualiza o dashboard.

4.2 Gestão de Sensores — Estados, Heartbeats e Notificações

Cada sensor é validado no momento da ligação por consulta a um ficheiro CSV de registo. Após validação, o sensor pode transitar entre quatro estados: ativo, inativo, manutenção e desativado. O Gateway monitoriza periodicamente os heartbeats recebidos e marca como inativo qualquer sensor que ultrapasse o intervalo máximo sem sinal. As notificações de bateria baixa e carregamento são enviadas fora de banda através da ligação de controlo TCP, permitindo que o Gateway registe e reaja a eventos operacionais sem interferir com o fluxo de medições.

4.3 Persistência e Dashboard

O Servidor mantém dois mecanismos de persistência complementares: os dados estruturados são inseridos numa base de dados SQLite (com controlo de acesso concorrente por semáforo) e os dados brutos são guardados em ficheiros separados por tipo. Em caso de falha temporária no envio ao Servidor, o Gateway preserva os lotes em ficheiros pending, assegurando tolerância parcial a falhas. O dashboard, disponível em `localhost:8080`, apresenta medições recentes, estatísticas por sensor e tipo, gráficos temporais, análises RPC, streams de vídeo e filtros interativos.

5. Resultados e Validação

O sistema foi testado com múltiplos sensores a operar em simultâneo. Os cenários de teste abrangeram: registo e validação de sensores por CSV; publicação de medições em diferentes tópicos RabbitMQ e confirmação da entrega ao Gateway; invocação do PreProcessamentoRPC e verificação dos valores normalizados; agregação de medições em lotes e envio ao Servidor via TCP; persistência em SQLite e em ficheiros; invocação do AnaliseRPC após receção de lote e visualização das estatísticas geradas; e atualização em tempo quase-real do dashboard com medições, gráficos e análises. Foram ainda testados cenários de heartbeat (detecção de sensor inativo), notificações de bateria e encaminhamento de stream de vídeo até ao dashboard.



Figura 2 — Dashboard web do sistema OneHealth com dados em tempo quase-real.

```
PS C:\Users\lucas\Source\Repos\ProjetoSD\PreProcessamentoRPC> python .\preprocessamento_rpc.py
[RPC PRE] Servico de pre-processamento ativo na porta 7000.
[RPC PRE] Formato: PREPROCESS|gateway_id|sensor_id|timestamp|tipo|valor
[RPC PRE] GW01 pediu S101/TEMP: 31.5 -> 31.5
[RPC PRE] GW02 pediu S101/TEMP: 31.5 -> 31.5
[RPC PRE] GW01 pediu S101/HUM: 50 -> 50
[RPC PRE] GW01 pediu S101/RUIDO: 64 -> 64
[RPC PRE] GW01 pediu S102/TEMP: 26.9 -> 26.9
[RPC PRE] GW02 pediu S102/TEMP: 26.9 -> 26.9
[RPC PRE] GW01 pediu S102/PM2.5: 14.5 -> 14.5
[RPC PRE] GW01 pediu S102/RUIDO: 87 -> 87
[RPC PRE] GW02 pediu S101/TEMP: 32.6 -> 32.6
[RPC PRE] GW01 pediu S101/TEMP: 32.6 -> 32.6
[RPC PRE] GW01 pediu S101/HUM: 86 -> 86
[RPC PRE] GW01 pediu S101/RUIDO: 55 -> 55
[RPC PRE] GW02 pediu S102/TEMP: 29.5 -> 29.5

PS C:\Users\lucas\Source\Repos\ProjetoSD\AnaliseRPC> python .\analise_rpc.py
[RPC ANALISE] Servico de analise ativo na porta 7001.
[RPC ANALISE] Formato: ANALYZE_BATCH|gateway_id|n_registos ... END
[RPC ANALISE] GW01: 14 registos, media=S101/HUM=55.25; S101/RUIDO=60.75; S101/TEMP=26.28, risco=MEDIO
[RPC ANALISE] GW01: 6 registos, media=S102/PM2.5=28.3; S102/RUIDO=82; S102/TEMP=28.2, risco=ALTO
[RPC ANALISE] GW01: 2 registos, media=S101/HUM=89; S101/RUIDO=60, risco=MEDIO
[RPC ANALISE] GW02: 1 registos, media=S101/TEMP=17.1, risco=BAIXO
[RPC ANALISE] GW02: 1 registos, media=S102/TEMP=24.3, risco=BAIXO
[RPC ANALISE] GW01: 2 registos, media=S101/HUM=51; S101/RUIDO=82, risco=MEDIO
[RPC ANALISE] GW01: 2 registos, media=S102/PM2.5=26.1; S102/RUIDO=90, risco=ALTO
```

Figura 3 — Execução simultânea dos componentes do sistema.

6. Conclusão

O nosso projeto OneHealth demonstrou que a combinação de Pub/Sub, RPC e sockets TCP permite construir um sistema distribuído modular, observável e com separação clara de responsabilidades. A adoção do RabbitMQ eliminou o acoplamento direto entre Sensores e Gateway, enquanto os serviços RPC em Python permitiram segregar a lógica de validação e análise sem sobrecarregar os componentes principais. A persistência dupla (SQLite e ficheiros) e o mecanismo de pending garantem resiliência básica face a falhas transitórias.

Como trabalho futuro, identificam-se as seguintes melhorias prioritárias: autenticação e autorização entre componentes; deploy em máquinas físicas separadas para validação de latência e tolerância a falhas reais; substituição do mecanismo de pending por uma fila de mensagens persistente; métricas em tempo real no dashboard (WebSockets ou Server-Sent Events); e melhoria da interface gráfica com alertas visuais e filtros avançados.